

Настройка и установка на сервере OpenVPN



Оглавление

- Настройка и установка на сервере OpenVPN
 -  Оглавление
- ⚠ Проблема
- 🎯 Цель
- Решение
 - Проверка доступных портов и протоколов
 - Открываем порты на нашем сервере VPN
 - проверяем возможные подключения

- проверяем пакеты UDP
- выясняем порт на котором будет строить VPN
- Установка VPN
 - Подготовка сервера
 - Создание PKI (сертификаты/ключи) — quickstart с easy-rsa
- Сгенерируйте клиентские ключи (пример для клиента client1):
 - Скопируйте нужные файлы в /etc/openvpn/server:
- Пример server.conf (UDP 1194 — дефолт)
 - Полный разбор конфигурации OpenVPN (каждая строка)

- Включаем IP forwarding (маршрутизация)
 - открываем файл конфигурации
 - `net.ipv4.ip_forward=1`
- Настройки фаервола (важно)
 - Разрешаем SSH и OpenVPN (tcp 3000) и включаем UFW:
 - Разрешаем UFW форвардить пакеты
 - Перезагружаем UFW
 - Запуск и автозапуск OpenVPN
- Генерация клиентского .ovpn профиля
 - Заголовок конфигурации клиента
 - `dev tun`
 - `proto tcp`

- remote YOUR_SERVER_IP
3000
- resolv-retry infinite
- nobind
- persist-key
- persist-tun
- remote-cert-tls server
- cipher AES-256-CBC
- verb 3
- Вставка файлов
сертификатов и ключей
- OpenVPN позволяет
вставить содержимое
прямо в конфиг:
- key-direction 1
- ИТОГ по вставке файлов
- Проверки после запуска
 - На клиенте
 - На сервере
- Итог

Проблема

В отеле где сейчас нахожусь
зарезаны все сомнительные
порты udp и просто даже
поиграть в SteamDec
невозможно. но доступ к веб
страницам разрешено



Цель

- Выяснить диапазон разрешенных сетей и протоколов
- Выйти из железного занавеса
- Универсально подключить все устройства к VPN
- Минимальные настройки на клиенте

- Минимальные настройки на сервере
-

Решение

Проверка доступных портов и протоколов

Открываем порты на нашем сервере VPN

- список портов для проверки (можно использовать свои но если работает белый список(все что не разрешено запрещено) то лучше использовать их)
- Telegram использует 443 так что именно по этому его не смогли в свое время заблокировать потому что этот порт еще используют все сайты

мира но для звонков он использует
другой протокол и возможно порт
так что он может не работать

Порт	Протокол-
Назначение	
Использование	
Примечание	

🌐 Веб и HTTPS

53	UDP/TCP DNS
----	-------------

✗ не использовать

почти всегда открыт

80	TCP HTTP
----	----------

✓ можно использовать

почти всегда открыт

443	TCP/UDP HTTPS
-----	---------------

✓ можно использовать

почти всегда открыт

8080	TCP/UDP HTTP
------	--------------

alternate ✓ можно

использовать часто открыт
8443 TCP/UDP HTTPS
alternate ✓ можно
использовать часто открыт
8888 TCP/UDP HTTP
alternate ✓ можно
использовать часто открыт

✉ Почтовые сервисы

25 TCP SMTP
(отправка почты) ⚠
нежелательно
часто блокируется
провайдерами
465 TCP SMTPS (SMTP
over SSL) ✓ можно
использовать защищённая
отправка
587 TCP SMTP
(submission) ✓ можно
использовать стандарт для
клиентов
110 TCP POP3

✓ можно использовать
получение почты
995 TCP POP3S (POP3
over SSL) ✓ можно
использовать защищённый
POP3
143 TCP IMAP
✓ можно использовать
получение почты
993 TCP IMAPS (IMAP
over SSL) ✓ можно
использовать защищённый
IMAP

 VPN и прокси

1194 UDP/TCP OpenVPN
✓ можно использовать
дефолтный порт OpenVPN
500 UDP IKE (IPSec)
✓ можно использовать для
L2TP/IPSec
4500 UDP NAT-T
(IPSec) ✓ можно

использовать для VPN

через NAT

1701 UDP L2TP

⚠ с IPSec только

редко используется напрямую

1723 TCP PPTP

⚠ устарел

часто блокируется

51820 UDP WireGuard

✓ можно использовать

современный VPN

1080 TCP SOCKS proxy

✓ можно использовать для

прокси-сервисов

📱 Удалённое управление

22 TCP/UDP SSH

✗ не использовать

часто мониторится

23 TCP Telnet

✗ не использовать

небезопасен

3389 TCP RDP

⚠ осторожно

часто блокируется

5900 TCP VNC

⚠ осторожно

часто открыт в LAN

25565 TCP Minecraft
server ✓ можно

использовать часто

открыт, не подозрителен

 Файловые сервисы

20, 21 TCP FTP

⚠ нежелательно

небезопасен без TLS

989, 990 TCP FTPS

(secure FTP) ✓ можно

использовать безопасный

FTP

137–139 TCP/UDP NetBIOS

✗ не использовать

только для LAN

445 TCP SMB

✗ не использовать

опасен в WAN

2049 TCP/UDP NFS

⚠️ осторожно

используется в Unix-сетях

873 TCP rsync

✓ можно использовать
синхронизация файлов

 Прочие полезные

123 UDP NTP (время)

✓ можно использовать
почти всегда открыт

161–162 UDP SNMP

⚠️ осторожно для
мониторинга устройств

3306 TCP MySQL

⚠️ не выставлять наружу
только локально

5432 TCP PostgreSQL

⚠️ только внутри сети
только локально

6379 TCP Redis

⚠️ только внутри сети

ТОЛЬКО локально

27017 TCP MongoDB

⚠ не наружу

ТОЛЬКО локально

🎮 Игровые сервисы

25565 TCP Minecraft

✓ можно использовать

открыт часто, подходит под
маскировку

19132-19133 UDP Minecraft

Bedrock ✓ можно

использовать мобильные/
консольные клиенты

27015 UDP/TCP

Steam(CS, TF2...) ✓ можно

использовать открыт на
многих сетях

7777 UDP Terraria

✓ можно использовать

часто открыт

9987 UDP TeamSpeak 3

voice ✓ можно

использовать подходит для
UDP трафика

16261 UDP Project

Zomboid ✓ можно

использовать редко

блокируется

50000–65535 UDP Игры EA...

✓ можно использовать

динамические игровые
диапазоны

3478–3480 UDP/TCP

STUN/PS... ✓ можно

использовать часто

разрешены для голосовых

 VoIP, мессенджеры,
конференции

3478 UDP/TCP (Zoom,
Discord) ✓ можно

использовать почти всегда
открыт

5349 TCP (TURN,
WebRTC secure) ✓ можно

использовать альтернатива
3478

19302–19309 UDP Google STUN
servers ✓ можно

использовать популярные и
не блокируются

5004–5005 UDP(аудио/видео
потоки) ✓ можно

использовать базовый VoIP
диапазон

10000 UDP Jitsi /
WebRTC media ✓ можно

использовать редко
блокируется

5222 TCP Jabber
✓ можно использовать

обычно открыт

5223 TCP XMPP over
SSL ✓ можно

использовать защищённая
версия

3479 UDP Discord
voice backend ✓ можно

использовать аналог 3478

💬 Соцсети, мессенджеры,
вспомогательные

5228–5230 TCP/UDP
GooglePlay Services ✔ можно
использовать всегда
открыты

2195–2196 TCP Apple Push
Notification ✔ можно
использовать
используется iOS

443 TCP/UDP HTTPS/
✔ можно использовать
основной и безопасный

☁ Облачные, обновления, CDN

53 UDP DNS

⚠ осторожно
обычно открыт, но легко
палится

8080 TCP/UDP HTTP
alt ✔ можно

использовать под веб-трафик

8888 TCP/UDP Google

DNS ✓ можно

использовать не вызывает подозрений

53, 853 TCP/UDP DNS /

DoT ✓ можно

использовать иногда подходит для обхода

500, 4500 UDP IPSec

✓ можно использовать VPN безопасный канал

 Редкие, но полезные порты для обхода / маскировки

4443 TCP/UDP HTTPS

alt ✓ можно

использовать выглядит как корпоративный SSL

2083 TCP cPanel SSL

/ WebDAV ✓ можно

использовать	редко
блокируется	
2053	TCP/UDP
Cloudflare DNS	✓ можно
использовать	часто
открыт	
20000	TCP cPanel
WebDAV	✓ можно
использовать	не
блокируется	
123	UDP NTP (время)
✓ можно использовать	
фоновый легитимный трафик	

- идеальный пример если у вас есть доменное имя подключим его к своему VPS выставляем 443 для OpenVPN и все любая сеть видит (да все действия отслеживаются vpn работает по методу туннеля, трубу видно что внутри нет но от куда и куда идет труба видно) что мы

подключены к доменному имени `www.example.com` и по протоколу TCP просматриваем (скорее всего HTML) мы же на порту 443 а то что много так сейчас сайты во какие тяжёлые и паршивые и не оптимизированные.

проверяем возможные подключения

- Первое предлагаю определить можем ли мы вообще достучаться до нашего сервера с доменным именем или без
- s. Сервер(возможно ПК отдельный терминал)

```
sudo apt install netcat
```

устанавливаем netcat ([его документация онлайн](#))

```
sudo sh -c 'while true; do  
echo -e "HTTP/1.1 200  
OK\r\nContent-Type:  
text/plain\r\n\r\nOK" | nc  
-l -p 80 -q 1; done'
```

- sudo - Даёт права суперпользователя. Порт 80 — это “привилегированный” порт (требуется root). Без sudo команда не сможет слушать этот порт.
- sh -c '...' - Запускает внутри sh (shell) мини-программу в одну строку, заключённую в кавычки. Нужно, потому что конструкция while ...; do

...; done — это синтаксис шелла, а sudo без sh -с не поймёт его.

- while true; do ...; done Бесконечный цикл: while true — пока условие "истинно" (всегда). Внутри него находится блок, который выполняется при каждом новом соединении. Когда ты нажмёшь Ctrl + C, цикл прервётся.
- echo -e "HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\nOK" Это формирует HTTP-ответ, который сервер отправляет клиенту.

Расшифровка:

```
HTTP/1.1 200 OK
← статус: всё хорошо
Content-Type: text/plain
← тип контента — обычный
```

текст

← (пустая строка отделяет
заголовки от тела)

OK

← тело ответа (содержимое)

Флаги:

-e — позволяет
использовать управляющие
последовательности (`\r`, `\n`
и т.д.).

`\r\n` — перевод строки по
стандарту HTTP.

- | nc -l -p 80 -q 1 - Это Netcat (nc) —
простая утилита для прослушивания
TCP-портов.

Параметры:

-l — слушать порт (listen mode).

-p 80 — номер порта (80).

-q 1 — закрыть соединение через 1 секунду после отправки данных (иначе может “зависнуть”).

- Разреши доступ через брандмауэр (Если включён ufw):

```
sudo ufw allow 80/tcp  
sudo ufw status
```

- | — пайп. Передаёт вывод echo на вход ps. То есть при подключении

клиент получает именно то, что вывел echo.

- и. Юзер (другое устройство подключенное к вайфаю отеля или просто второй терминал)

```
curl -i  
http://<IP_сервера>/
```

curl - Основная программа. Она “ходит” по URL и показывает ответ от сервера (аналог браузера, но в терминале).

Флаг:

```
-i “Покажи не только тело  
ответа, но и HTTP-  
заголовки” .
```


http://<IP_сервера>/ - URL, к которому ты обращаешься.

- Два варианта ответа :

1. Все ок

```
HTTP/1.1 200 OK  
Content-Type: text/plain
```

OK

2. Все не ок

```
curl: (7) Failed to connect  
to 192.168.1.10 port 80:  
Connection refused
```

Это значит, что:

- сервер не запущен,
- или порт 80 закрыт в firewall,
- или ты указал не тот IP.
- В моем случае я в Германии так что тут обращаться непосредственно по ip нельзя оно перебрасывает на страницу запрещено (так что в моем случае просто на уровне страны нужно доменное имя из плюсов нужно любое доменное имя хоть с тремя точками \$\$\$=.com \$\$=.что-то.com \$=что-то.что-то.com)

```
HTTP/1.1 404 Not Found
Pragma: no-cache
Connection: close
Content-type: text/html;
charset=utf-8
```

Content-Length: 5521

...

Но учитывая что у меня уже есть доменные имена я буду ими дальше пользоваться но на них есть сайты так что мне теперь становится важно их не сломать...так чтобы и сайт обычный в интернете и для знающего и держащего ключи это ВПН

- есть возможность настроить nginx правильное распределение и если проходит запрос с оглавлениями(так все браузеры делают) то отдаем сайт если нет то отдаем vpn но в данном случае не буду это рассматривать а просто нащупаю возможный порт если есть желание раскрыть эту тему

напишите мне я ее разберу
подробно насколько возможно.

Выяснили как вообще достучатся до сервера. просто что это вообще возможно. Есть сервера которые находятся в другом регионе и тогда вы до них не достучитесь(просто заблокированы) но вопрос как вы его арендовали... в моем случае можно.

проверяем пакеты UDP

пакеты tcp и udp отличаются тем что первый именно смотрит за точность доставленных данных и если что пошлет их еще раз(что грузит сеть) а второй отправляем и ему все равно получил ли ты или нет чаще использует видео и игры ибо потеря некоторых данных не катастрофична (просто зависнет видео,

пропадет часть звука) но так как я знаю что у меня upd в отеле зарезано советую с этого начать ибо вы избежите проблемы я все сделал почему не работает как у меня

- s. на сервере вешаем слушателя
 - не забываем про firewall если он есть

```
sudo nc -u -l -p 80
```

-u → UDP. Говорит nc работать по UDP вместо TCP (по умолчанию nc – TCP).

-l → listen – запустить в режиме прослушивания (сервер). nc ждёт входящих соединений/пакетов, а не

устанавливает исходящее подключение.

-p <порт> → port – указать локальный порт, на котором слушать (в режиме -l) или от которого исходить (в некоторых реализациях). Обычно используется при запуске в режиме -l для задания порта.
просто слушаем 80 порт

- к. на клиенте отправляем

```
echo -n "UDP TEST" | nc -u  
<IP_сервера> 80
```

просто отправляем текст UDP TEST на наш сервер в моем случае еще используя

доменное имя

в моем случае ничего не пройдет (да на отдельном wifi нельзя делать звонки) кроме некоторых мессенджеров которые все называют тормознувшими и я теперь знаю почему спасибо разности протоколов если у вас все ок то вы получили в терминале сервера "UDP TEST" вам повезло и у вас будет более быстрый ВПН чем у меня

выясняем порт на котором будет строить ВПН

по последней схеме открываем порты
[список в начале](#)

например 3000 порт(скорее всего не получится)

- внимание я уже работаю с просто tcp без флага -u
- не забываем про firewall если он есть

сервер

```
sudo nc -l -p 3000
```

клиент

```
echo -n "have connect" |  
nc <IP_сервера> 3000
```

- удачи повторять пока не найдешь нужный майнкрафт обычно работает но ходят слуги когда-то почему-то все не могли

подключатся к серверам
майнкарфта почему бы это....

Установка ВПН

- Я буду использовать порт 3000 но вам необходимо использовать тот который подходит именно вам
- Установка происходит на Ubuntu 22.04 как «типичный» сервер. Если у вас другая версия/дистрибутив — многие шаги похожи, но пакеты/команды могут отличаться.
Основные источники: официальная документация Ubuntu/OpenVPN и популярные руководства.(1)
[https://documentation.ubuntu.com/server/how-to/security/install-openvpn/?utm_source=chatgpt.com],
(2)[

install-for-linux.html?
utm_source=chatgpt.com], (3)
[https://www.digitalocean.com/comm
unity/tutorials/how-to-set-up-and-
configure-an-openvpn-server-on-
ubuntu-22-04?
utm_source=chatgpt.com]

Подготовка сервера

0. Обновление системы

```
sudo apt update && sudo apt  
upgrade -y
```

- флаг -y нужен чтобы отправить y(yes) во все места где система спрашивает точно уверены

1. Установите нужные пакеты
(OpenVPN +)

```
sudo apt install -y openvpn  
easy-rsa ufw
```

ufw - firewall easy-rsa - для генерации
ключей openvpn - OpenVPN сам пакет для
генерации ключей

Создание PKI (сертификаты/ключи)
— quickstart с easy-rsa

1. Перейдём в каталог easy-rsa и
инициализируем PKI:

```
make-cadir ~/openvpn-ca
```

- создаёт новый каталог с шаблонной структурой PKI (сертификатного центра)

```
cd ~/openvpn-ca
```

- зайти в каталог - далее буду игнорировать такие простые команды

```
./easyrsa init-pki
```

- команда (аргумент) для easyrsa. Означает инициализировать PKI (Public Key Infrastructure) — создает структуру каталогов и служебные файлы для хранения CA, запросов

(CSRs), выданных сертификатов и приватных ключей.

- `./` — указание оболочке запустить файл `easysga` из текущего каталога. Без `./ shell` будет искать в `$PATH`.
- `easysga` — название исполняемого файла/скрипта Easy-RSA v3.
- `init-pki` — команда (аргумент) для `easysga`. Означает инициализировать PKI (Public Key Infrastructure) — создает структуру каталогов и служебные файлы для хранения CA, запросов (CSRs), выданных сертификатов и приватных ключей.

- ```
pki/
└─ private/ #
```

приватные ключи (доступ  
только владельцу)

└─ issued/ #

подписанные сертификаты

└─ reqs/ #

запросы на подпись  
(CSR)

└─ index.txt #

база данных  
сертификатов

└─ serial #

файл с текущим серийным  
номером

└─ ...

- Никаких сертификатов ещё нет — только «пустая» структура.

```
./easyrsa build-ca nopass
```

- подпишет CA (без пароля для простоты — в продакшене можно задать пароль)
- `build-ca` — подкоманда: создать собственный корневой центр сертификации (CA). В процессе появится корневой сертификат `ca.crt` и приватный ключ CA `private/ca.key`.
- `porass` — опция, означающая не шифровать приватный ключ CA паролем (т.е. не запрашивать `passphrase` при создании/использовании ключа).
- Если указать без `porass`, Easy-RSA спросит пароль и зашифрует `ca.key` паролем; при подписи сертификатов нужно будет вводить пароль. `porass` удобен для

автоматизации, но менее безопасен: приватный ключ СА хранится в виде обычного файла и при компрометации сервера злоумышленник сможет подписывать произвольные сертификаты.

Результат:

```
pki/ca.crt —
корневой сертификат СА
(публичный)

pki/private/ca.key —
приватный ключ СА (если
nopass — в открытом виде)
```

- Важно по безопасности: приватный ключ СА — самая чувствительная



вещь. Доступ к нему должен быть строго ограничен, например `chmod 600 pki/private/ca.key` и запускать команды в безопасной среде.

```
./easysrsa gen-req server
nopass
```

- `gen-req` — «generate request» — сгенерировать запрос на подпись (CSR) и приватный ключ. CSR (Certificate Signing Request) — файл, который содержит публичную часть + метаданные (Common Name и др.) и отправляется СА для подписи.
- `server` — имя запроса / Common Name (CN). Обычно указывают `server` для серверного сертификата, но это может быть любое строковое

имя (например `vpn.example.com` или `host1`). Это имя позже используется при подписании (оно станет именем итогового сертификата `issued/server.crt`).

- `porpass` — опять означает не защищать приватный ключ паролем. Тогда приватный ключ будет в `pki/private/server.key` и никак не зашифрован.

Что создаётся:

```
pki/private/server.key
— приватный ключ для
объекта с именем server.
(Чувствительный файл —
chmod 600.)
```

```
pki/reqs/server.req (или
pki/reqs/server.csr) —
```

CSR, который потом подпишет  
CA.

- Примечание: если хотите, чтобы приватный ключ был зашифрован паролем, не используйте `porass` — тогда вам будет предложено ввести `passphrase`.

```
./easyrsa sign-req server
server
```

- `sign-req` — команда подписать запрос (CSR).
- Первый аргумент `server` — тип запроса. В Easy-RSA возможны типы `server` или `client` (и иногда `ca/user` в зависимостях). Тип влияет на

расширения X.509 (например `server` обычно получает `extendedKeyUsage=serverAuth`, а `client` — `clientAuth`). Это важно, чтобы OpenVPN/SSL знали, для чего этот сертификат предназначен.

- Второй аргумент `server` — имя запроса, то есть имя CSR, которое вы указали при `gen-req`. Если CSR назывался `server`, используем `server`. Можно было бы `sign-req client client1` для подписания клиентского CSR `client1`.

Что происходит:

Easy-RSA берет `pki/reqs/server.req`, подписывает его своим CA (`pki/private/ca.key`) и записывает результат в `pki/issued/server.crt`.

Результат:

pki/issued/server.crt — подписанный сертификат сервера, используемый OpenVPN.

pki/private/ca.key — используется для подписи (не раскрывается).

Замечание: команда может запросить подтверждение (Are you sure?) и/или пароль CA, если CA был создан с паролем.

```
./easyrsa gen-dh
```

- gen-dh — сгенерировать параметры Diffie-Hellman (DH). Это большой файл (в зависимости от размера ключа, например 2048/4096 бит) который используется при

установлении защищённого канала TLS с RSA/PKI. OpenVPN в классической конфигурации использует DH параметры для согласования ключей.

Что создаётся:

pki/dh.pem (или pki/dh-XXXX.pem) — файл с DH-параметрами, копируется затем в /etc/openvpn/ как dh.pem.

Примечание по современной криптографии:

DH (DHE) — традиционный способ. В современных установках можно использовать ECDH (эллиптические кривые) — они короче и быстрее. Easy-RSA поддерживает и ECDH/EC ключи, но gen-dh генерирует именно классические DH параметры.

```
openvpn --genkey --secret
ta.key
```

- `openvpn` — запуск исполняемого файла OpenVPN (в данном случае утилита встроена в пакет).
- `--genkey` — опция: генерировать симметричный (статический) ключ.
- `--secret ta.key` — указать имя выходного файла, куда сохранится сгенерированный ключ (в данном примере `ta.key`).

Что это делает:

Создаёт TLS-auth / HMAC ключ (предшарированный секрет), который используют в опции OpenVPN `tls-auth` (или `tls-crypt`).

tls-auth добавляет дополнительную HMAC-защиту к управляющим пакетам TLS (помогает блокировать неавторизованные подключения и некоторые типы DoS).

В конфиге сервера обычно прописывают `tls-auth /etc/openvpn/ta.key 0`, а в клиентском `tls-auth ta.key 1` (последняя цифра — `key-direction`, объяснение ниже).

Файл `ta.key` — простой бинарный/текстовый файл с симметричным ключом. Должен быть одинаковым и на сервере, и на клиенте.

Про `key-direction`:

При использовании `tls-auth` сервер ставит `key-direction 0` (в `server.conf`), а клиент — `key-direction 1` (в `.ovpn`). Это указывает, в каком направлении используется HMAC-



ключ. Если забыть выставить — будет отказ в подключении.

Альтернатива/замена:

tls-crypt — более новая опция, шифрующая и защищающая управляющие пакеты одним ключом; `openvpn --genkey --secret` также используется для генерации ключа, но использование конфигурации отличается. `tls-crypt` удобнее (скрывает метаданные) — подумайте о ней как о варианте.

Сгенерируйте клиентские ключи (пример для клиента `client1`):

```
./easyrsa gen-req client1
nopass
```

Разделим на токены:

- ./ — запустить исполняемый файл из текущей директории.
- easyrsa — сам скрипт Easy-RSA v3.
- gen-req — команда: generate request — сгенерировать пару ключей и создать CSR (Certificate Signing Request).

Что делает: генерирует приватный ключ и соответствующий ему запрос (CSR), который содержит публичный ключ и набор полей (CN, O, OU и т.д.), которые будут подписаны СА.

- client1 — имя (Common Name, CN) для создаваемого ключа/запроса. Это имя станет меткой: приватный ключ будет лежать как

pki/private/client1.key, CSR как pki/reqs/client1.req (или client1.csr). При подписании вы получите pki/issued/client1.crt. Поэтому имя должно быть уникальным среди уже выданных сертификатов/запросов — не делайте два клиента с одинаковым CN, иначе путаница и возможные проблемы с отзывом.

- `porass` — опция: не шифровать приватный ключ паролем. В результате приватный ключ pki/private/client1.key будет храниться в открытом виде (PEM, без защиты). Удобно для автоматизации и чтобы клиент мог подключаться без ввода пароля, но это менее безопасно: если файл украдут — злоумышленник сразу сможет подключиться под этим

клиентом или извлечь приватные данные.

```
./easyrsa sign-req client
client1
```

Опять разбираем:

- `./easyrsa` — запускаем тот же скрипт.
- `sign-req` — команда: подписать запрос (take a CSR and sign it with the CA). Это превращает CSR в полноценный сертификат X.509, подписанный вашим CA.
- `client` — тип запроса. Easy-RSA использует типы `server` или `client`. Тип важен, потому что при генерации конечного сертификата в

него будут включены  
определённые расширения X.509:

- server → extendedKeyUsage = serverAuth (сертификат предназначен для сервера),
- client → extendedKeyUsage = clientAuth (сертификат предназначен для клиента).  
OpenVPN и TLS смотрят на эти расширения. Если вы подписываете клиентский запрос, указывайте client.
- client1 — имя/идентификатор CSR, то есть то имя, которое вы использовали в gen-req. Easy-RSA найдёт pki/reqs/client1.req, подпишет его CA и положит результат в pki/issued/client1.crt.

Что происходит внутри:

Easy-RSA читает `pki/reqs/client1.req`.

Подписывает запрос приватным ключом  
CA (`pki/private/ca.key`), используя текущие  
политики/параметры (размер ключа, срок  
годности, `extensions`).

Обновляет базу данных CA: `pki/index.txt`  
(вносит запись о выданном сертификате),  
инкрементирует `pki/serial` и т.д.

Создаёт файл `pki/issued/client1.crt`.

Куда положится результат

```
pki/issued/client1.crt #
подписанный сертификат
клиента
```

## Если СА защищён паролем

Если вы создавали СА с паролем (не использовали `pass` для СА), при `sign-req` Easy-RSA спросит `passphrase` СА (чтобы использовать `pki/private/ca.key` для подписи). Это повышает безопасность — `private CA-key` не используется «в открытом виде» автоматически.

Проверка результата:

```
посмотреть cert
openssl x509 -in
pki/issued/client1.crt -
noout -text

сверить subject и
extensions
openssl x509 -in
pki/issued/client1.crt -
```

```
noout -subject -dates -ext
extendedKeyUsage
```

Скопируйте нужные файлы в  
/etc/openvpn/server:

```
sudo cp pki/ca.crt
pki/issued/server.crt
pki/private/server.key
pki/dh.pem ta.key
/etc/openvpn/server/
```

- все файлы(**pki/ca.crt...**) будут  
скопированы(**cp**) внутрь последней  
папки(**etc/openvpn/server/**)

Пример server.conf (UDP  
1194 — дефолт)



Создайте /etc/openvpn/server/server.conf со следующим содержимым (пример минимальный, под IPv4 и NAT):

```
port 3000 #НЕ верный порт у
вас свой!
proto tcp
dev tun
ca
/etc/openvpn/server/ca.crt
cert
/etc/openvpn/server/server.
crt
key
/etc/openvpn/server/server.
key
dh
/etc/openvpn/server/dh.pem
tls-auth
/etc/openvpn/server/ta.key
0
topology subnet
```

```
server 10.8.0.0
255.255.255.0
push "redirect-gateway def1
bypass-dhcp"
push "dhcp-option DNS
1.1.1.1"
keepalive 10 120
cipher AES-256-CBC
user nobody
group nogroup
persist-key
persist-tun
status /var/log/openvpn-
status.log
verb 3
```

- Используйте *proto udp* на *proto tcp* и при необходимости.

Полный разбор конфигурации  
OpenVPN (каждая строка)

1. port 3000 #НЕ верный порт у вас свой!

port — директива задаёт порт, на котором сервер слушает входящие соединения.

3000 — номер порта в примере. Обычно OpenVPN использует 1194 (UDP) по умолчанию; вы можете выбрать любой порт, но:

если порт нестандартный —  
прокомментируйте это в документации и  
откройте его в фаерволе;

некоторые провайдеры/фильтры  
блокируют нестандартные порты.

Комментарий #НЕ верный порт у вас свой!  
— просто напоминание: подставьте ваш  
фактический публичный порт.

Практика: если хотите «маскировать» VPN как HTTPS, можно использовать порт 443 (TCP), но это имеет свои trade-offs (см. proto).

## 2. proto tcp

proto указывает транспортный протокол: tcp или udp.

tcp — сервер будет слушать TCP.

Особенности:

соединения над TCP надёжные (встроенная доставка/повторная пересылка), но менее производительны для VPN (двойная ретрансляция: TCP внутри TCP) — возможны задержки при потере пакетов;

легче обходить некоторые сетевые ограничения (firewall/проxy), т.к. TCP-порт

443 часто открыт.

Если производительность важнее — обычно рекомендуют `proto udp`. Решение зависит от ваших требований.

### 3. `dev tun`

`dev` задаёт виртуальный сетевой интерфейс, который создаёт OpenVPN.

`tun` — TUN (уровень IP) — = routed IP-tunnel; пересылает L3 IP-пакеты.

Альтернатива: `dev tap` — мостовой Ethernet-интерфейс (L2), нужен если требуется передача широковещательных/NetBIOS трафика или если вы собираетесь бриджить(соединять) VPN с локальной сетью. `tap` громоздче и обычно не нужен для обычного доступа в интернет.

#### 4. `ca /etc/openvpn/server/ca.crt`

`ca` — путь к сертификату центра сертификации (CA), публичной части. OpenVPN использует его чтобы проверять присланные клиентские сертификаты.

`/etc/openvpn/server/ca.crt` — где ожидается файл. Файл публичный (можно раздавать клиентам) — но храните исходный CA в защищённом месте.

#### 5. `cert /etc/openvpn/server/server.crt`

`cert` — путь к сертификату сервера (подписанному CA). Этот файл содержит публичный ключ сервера и данные X.509.

OpenVPN будет использовать его при установке TLS-соединения.

#### 6. `key /etc/openvpn/server/server.key`

key — приватный ключ сервера (PEM).  
Крайне чувствительный файл — должен храниться только на сервере.

Важные действия: `chmod 600 /etc/openvpn/server/server.key` и владелец `root:root`. Если ключ скомпрометирован — нужно отзывать сертификат CA и пересоздавать.

## 7. `dh /etc/openvpn/server/dh.pem`

`dh` — путь к файлу параметров Diffie–Hellman (DH), который используется при установлении ключей (классический DHE).

Файл обычно создаётся `./easyrsa gen-dh` и копируется в `/etc/openvpn/....`

Примечание: DH-файл может быть большим (2048/4096 бит). Современные варианты используют ECDH (короче и

быстрее), но `dh.pem` — традиционный подход.

8. `tls-auth /etc/openvpn/server/ta.key 0`

`tls-auth` включает дополнительную HMAC-проверку пакетов TLS, используя пред-шарированный симметричный `ta.key`. Это защищает от неавторизованных/помеховых пакетов (защита от некоторых DoS/сканирования).

`/etc/openvpn/server/ta.key` — файл, сгенерированный командой `openvpn --genkey --secret ta.key`.

0 — `key-direction`: значение для сервера. Клиенты должны использовать `key-direction 1`. (Это указывает направление HMAC.)



Важно: `ta.key` нужно раздать всем клиентам и хранить в безопасности.  
Альтернатива: `tls-crypt` (один ключ шифрует и аутентифицирует метаданные) — может быть предпочтительнее.

## 9. topology subnet

`topology` задаёт способ выдачи IP клиентам.

`subnet` — классическая подсеть: сервер `server 10.8.0.0 255.255.255.0` выдаёт каждому клиенту индивидуальный IP в этой подсети. Эта топология проще и совместима с современными клиентами.

Альтернатива: `net30` (старый метод OpenVPN для `dev tun`), даёт каждой паре (сервер–клиент) /30 сеть, менее удобна. Для большинства случаев `subnet` предпочтительнее.

10. server 10.8.0.0 255.255.255.0

server комбинирует создание интерфейса и пул IP-адресов.

10.8.0.0 — сеть, которая будет выдана клиентам; 255.255.255.0 — маска сети (/24).

Это значит: диапазон 10.8.0.1..10.8.0.254 — доступен для клиентов (адрес сервера обычно 10.8.0.1). Всего ~254 клиентских адреса.

Выберите сеть, которая не конфликтует с вашей локальной сетью или сетью клиента. Если конфликт есть — используйте другой пул (например 10.9.0.0/24 или 172.16.x.0/24).

11. push "redirect-gateway def1 bypass-dhcp"

push — сервер отправляет клиенту опции маршрутизации/конфигурации при подключении.

"redirect-gateway def1 bypass-dhcp" — перенаправляет весь клиентский интернет-трафик через VPN. Что делает конкретно:

def1 — добавляет две маршрутизирующие записи (0.0.0.0/1 и 128.0.0.0/1) вместо замены полного маршрута по умолчанию; это более безопасный способ перехвата всего трафика, особенно при наличии DHCP/локальных маршрутных настроек.

bypass-dhcp — пытается не ломать локальный DHCP (на некоторых системах сохраняет доступ к локальному DHCP сервера).

Если вы НЕ хотите, чтобы весь трафик клиентов шёл через VPN, не используйте эту строку; вместо этого можно пробрасывать только нужные подсети через `push "route ..."`.

## 12. `push "dhcp-option DNS 1.1.1.1"`

`push "dhcp-option DNS X.X.X.X"` — сервер сообщает клиенту, какой DNS использовать. В данном примере это 1.1.1.1 (Cloudflare).

Можно указывать несколько DNS, например `push "dhcp-option DNS 1.1.1.1"` и `push "dhcp-option DNS 8.8.8.8"`.

Важно: на некоторых клиентах (особенно Linux) нужно дополнительно настроить `resolvconf/NetworkManager` для автоматической подстановки DNS.

### 13. `keepalive 10 120`

`keepalive` — удобная сокращённая директива, эквивалент для двух опций: `ping` и `ping-restart`. Формат: `keepalive .`

`10` — каждые 10 секунд OpenVPN посылает пинг сервер/клиенту.

`120` — если в течение 120 секунд нет пинга от противоположной стороны, соединение считается "упавшим" и будет перезапущено.

Эквивалент: `ping 10` и `ping-restart 120`.

Полезно для обнаружения разорванных соединений и автоматического восстановления.

### 14. `cipher AES-256-CBC`

cipher задаёт симметричный шифр для трафика TLS/data (алгоритм шифрования для канала).

AES-256-CBC — классический AES в режиме CBC с 256-битным ключом. Работает, но есть более современные варианты (например AES-GCM modes: AES-256-GCM) — GCM сочетает шифрование и аутентификацию и обычно быстрее/безопаснее.

Важно: клиенты и сервер должны поддерживать и согласовать один и тот же шифр. Можно использовать --data-ciphers в новых версиях OpenVPN, чтобы перечислить набор и дать приоритет современным.

15. user nobody

user указывает системного пользователя, под которым OpenVPN будет работать после инициализации и открытия привилегированных портов/сокетов.

nobody — стандартный неполномочный системный аккаунт. Эта директива снижает риск: OpenVPN запускается как root сначала (чтобы открыть порт/создать интерфейс), а затем опускает привилегии к nobody.

Убедитесь, что nobody существует в системе; альтернативы: openvpn или vpnuser (создайте специального пользователя для лучшей изоляции).

#### 16. group nogroup

group указывает группу, к которой будет переключён процесс после user. nogroup

— соответствующая группа для nobody в Debian/Ubuntu.

Аналогично: можно создать отдельную группу `openvpn/vpn` и дать минимальные права.

### 17. `persist-key`

`persist-key` — говорит OpenVPN не удалять/перегенерировать приватные ключи и связанные данные при перезапуске/рневале TLS-сессии. Это предотвращает необходимость заново считывать/вводить ключи при кратковременных рестартах.

Полезно в сочетании с `user/group`, чтобы ключи не терялись при смене привилегий.

### 18. `persist-tun`



`persist-tun` — не удалять виртуальный интерфейс `tun` при перезапуске OpenVPN. Оставляет интерфейс существующим и просто переподключает его при рестарте демона. Удобно для снижения прерываний.

#### 19. `status /var/log/openvpn-status.log`

`status` — файл, куда OpenVPN пишет статус текущих подключений (список типов `clients`, IP, время подключения, байты и т.д.).

`/var/log/openvpn-status.log` — путь в примере. Убедитесь, что процесс имеет права на запись в этот файл (после `user nobody` это означает, что либо файл создаётся `root` раньше, либо даются нужные права).

По этому файлу удобно смотреть, кто подключён (например `cat /var/log/openvpn-status.log`). Некоторые системы ротируют логи — настройте `logrotate` при необходимости.

## 20. `verb` 3

`verb` — уровень подробности логов (`verbosity`). Диапазон обычно 0..11 (0 минимально; 3 — обычный уровень для отладки без излишней болтовни).

`verb` 3 покажет полезную диагностическую информацию: успешные TLS-рукопожатия, ошибки аутентификации, выдачу IP и т.д. Для производственного режима часто ставят `verb` 1-3. Для детальной отладки временно повышают до `verb` 5+.

Не забывайте, что высокий `verb` может записывать чувствительную информацию

— не держите высокий уровень в проде.

## Включаем IP forwarding (маршрутизация)

открываем файл конфигурации

```
sudo nano /etc/sysctl.conf
```

- nano — команда для открытия файла в редакторе.
- /etc — папка с конфигурационными файлами.
- sysctl.conf — основной файл системных параметров ядра.

net.ipv4.ip\_forward=1

добавляем в самый конец/ или можно  
разкомментировать если такая строка/

проверяем до конца действует принцип кто последний тот и папа если в конце будет `net.ipv4.ip_forward=0` он и победит

```
net.ipv4.ip_forward=1
```

- Секция `net` — настройки сети ядра.
- Подсекция `IPv4`.
- `ip_forward` = «пересылка IP-пакетов»
- Значение:
  - 1 = включено
  - 0 = выключено

То есть:

`net.ipv4.ip_forward=1` = разрешить серверу перенаправлять трафик от одного интерфейса в другой.

[Клиент] ↔ (tun0) ↔  
[Сервер OpenVPN] ↔ (eth0)  
↔ [Интернет]

1. Он отправляет пакет → tun0
  2. Сервер принимает пакет → смотрит маршрут
  3. Перенаправляет пакет из tun0 → eth0 в интернет
  4. Интернет отправляет ответ → приходит в eth0
  5. Сервер перенаправляет ответ из eth0 → tun0
  6. Клиент получает ответ в VPN
- Если forwarding выключен — шаги 3 и 5 невозможны.
  - Сохранить:

```
Ctrl+O → Enter → Ctrl+X
```

### 3. Применить настройки

```
sudo sysctl -p
```

- `sysctl` — утилита для управления параметрами ядра Linux.
- Флаг `-p` = прочитать файл конфигурации (`/etc/sysctl.conf`) и применить параметры.

## Настройки фаервола (важно)

- всюду были решения окрой фаервол я предположу что это важно хотя у меня были проблемы с

любым шагом кроме фаервола  
просто открыли порт и все один раз

- но это мой опыт а ваш может отличаться

Разрешаем SSH и OpenVPN (tcp 3000) и включаем UFW:

```
sudo ufw allow OpenSSH
sudo ufw allow 3000/tcp
```

- `u f w` — Uncomplicated Firewall, простая оболочка над `iptables/nftables` для настройки фаервола.
- `allow` — разрешить трафик.
- `OpenSSH` — предопределённый сервис (обычно порт TCP 22), который `ufw` знает из `/etc/services` или профилей `ufw`.

Итого: открывает входящие соединения на порт SSH (22/tcp) для удалённого доступа к серверу. По нормальному вы сможете подключиться к ssh через ваш порт 10.8.0.1 конечно же когда подключены к VPN

```
ssh root@10.8.0.1
```

root - пользователь для примера (Вы же сделали отдельного пользователя чтоб рута заблочить ..... правда ведь? )

- 3000/tcp — порт 3000 протокола TCP.

```
START OPENVPN RULES
* nat
:POSTROUTING ACCEPT [0:0]
```



```
-A POSTROUTING -s
10.8.0.0/24 -o eth0 -j
MASQUERADE
COMMIT
END OPENVPN RULES
```

## 1. \*nat

\\* – указывает таблицу  
iptables.

nat – таблица Network  
Address Translation  
(перевод адресов для  
маршрутизации через  
интернет).

## 2. :POSTROUTING ACCEPT [0:0]

: – определение цепочки.

POSTROUTING – цепочка для обработки пакетов после маршрутизации, когда они выходят из сервера.

АССЕРТ – политика по умолчанию (принять).

[0:0] – счётчики пакетов и байтов (только статистика).

2. -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE

-A POSTROUTING – добавить правило в цепочку POSTROUTING.

-s 10.8.0.0/24 — источник пакетов (OpenVPN-сеть).

-o eth0 — интерфейс выхода в интернет (например, кабель или основной сетевой адаптер).

-j MASQUERADE — изменить исходный IP пакета на IP интерфейса eth0 (NAT), чтобы пакеты VPN клиентов могли выйти в интернет.

Итого: все пакеты с VPN сети (10.8.0.0/24), которые идут через eth0, будут “маскироваться” под IP сервера, чтобы работал интернет для клиентов.

### 3. COMMIT

Завершение блока правил для текущей таблицы (nat), применяет их.

Разрешаем UFW форвардить пакеты

```
sudo sed -i
's/DEFAULT_FORWARD_POLICY="
DROP"/DEFAULT_FORWARD_POLIC
Y="ACCEPT"/'
/etc/default/ufw
```

- sed — stream editor, инструмент для поиска и замены в файле.
- -i — редактировать файл на месте.
- 's/DEFAULT\_FORWARD\_POLICY="DROP"/DEFAULT\_FORWARD\_POLICY="ACCEPT"/'

CCEPT"/' — искать строку  
DEFAULT\_FORWARD\_POLICY="DROP"  
и заменить на ACCEPT.

- /etc/default/ufw — файл конфигурации UFW.
- если что можно по старинке (рассмотрено ранее через нано ручками все сделать)

Итого: разрешает UFW форвардить пакеты между интерфейсами, что обязательно для VPN.

## Перезагружаем UFW

```
sudo ufw disable
sudo ufw enable
sudo ufw status verbose
```

- `disable` — выключает UFW полностью.
- `enable` — включает UFW обратно.
- `verbose` — выводит детальную информацию: правила, политики по умолчанию, используемые интерфейсы.
- `reboot` - перезагрузить (не использовал так как могли выключить ранее а выключить и включить работает также)

Итого: перезагружаем UFW + позволяет проверить, что все порты открыты, политики форвардинга изменены, и VPN NAT работает.

## Запуск и автозапуск OpenVPN



```
sudo systemctl enable --now
openvpn-
server@server.service
sudo systemctl status
openvpn-
server@server.service
```

- systemctl - Команда для управления системными сервисами в systemd.
  - Через неё можно:
    - запускать сервисы
    - останавливать
    - перезапускать
    - включать автозапуск
    - проверять статус
- enable - Подкоманда systemctl.
  - Значение: включить автозапуск сервиса при старте

системы.

- То есть после перезагрузки OpenVPN будет запускаться автоматически.
- --now - Флаг для команды enable.
  - Значение: включить сервис прямо сейчас, не дожидаясь перезагрузки.
- То есть одновременно делает: enable + start.
- openvpn-server@server.service - Имя сервиса systemd.
  - Разбор:
    - openvpn-server — шаблон сервиса OpenVPN для

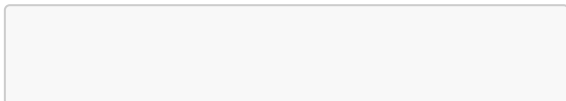


серверных  
конфигураций.

- @server — конкретная конфигурация, обычно файл  
/etc/openvpn/server/server.conf.
  - .service — это тип systemd юнита, показывает, что это сервис.
- Итого: Эта команда включает сервер OpenVPN сейчас и навсегда при старте системы, используя конфиг server.conf.
  - status - Подкоманда systemctl.
    - Показывает текущий статус сервиса:

- запущен / остановлен / ошибка
  - PID процесса
  - последние логи
- Итого: команда показывает жив ли OpenVPN сервер, работает ли правильно и нет ли ошибок.
- (имя сервиса зависит от размещения server.conf; в примере файл называется server.conf в /etc/openvpn/server/ — systemd unit openvpn-server@server). Проверьте лог /var/log/syslog или journalctl -u openvpn-server@server.

## Генерация клиентского .ovpn профиля



```
client
dev tun
proto tcp
remote YOUR_SERVER_IP 3000
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
cipher AES-256-CBC
verb 3
```

```
<ca>
...вставьте содержимое
ca.crt...
</ca>
<cert>
...вставьте client1.crt...
</cert>
<key>
...вставьте client1.key...
</key>
<tls-auth>
```

```
...вставьте ta.key...
</tls-auth>
key-direction 1
```

- Отлично, разберём каждую строку и каждое слово в клиентском конфиге OpenVPN, и сразу поясню про вставку файлов.

## Заголовок конфигурации клиента

```
client
```

- Это ключевое слово OpenVPN.
- Обозначает, что этот конфиг — для клиента, а не для сервера.
- OpenVPN будет запускаться в режиме клиента, а не слушать входящие подключения.

dev tun

dev — устройство. tun — виртуальный сетевой интерфейс уровня IP (туннель).

- OpenVPN создаёт виртуальный интерфейс tun0, через который будет идти весь VPN-трафик.
- Альтернатива: tap — Ethernet-уровень (уровень канала), нужен реже.

proto tcp

proto — протокол транспорта. tcp - Transmission Control Protocol , надежная доставка данных между компьютерами  
udp — User Datagram Protocol, быстрый, без проверки доставки.

- Можно UDP обычно лучше для VPN (меньше задержки).

`remote YOUR_SERVER_IP 3000`

`remote` — адрес сервера, к которому подключаться. `YOUR_SERVER_IP` — замените на IP или домен вашего OpenVPN сервера. `3000` — порт сервера (по умолчанию OpenVPN).

- Итого: клиент знает, куда идти.

`resolv-retry infinite`

`resolv-retry` — как часто пытаться разрешить DNS имя сервера. `infinite` — бесконечно, пока не получится соединиться.

`nobind`

Не привязывается к локальному порту. Клиент выбирает любой свободный порт на своей машине.

## `persist-key`

Не пересоздавать ключи при перезапуске соединения. Сохраняет сессии без повторного чтения ключей.

## `persist-tun`

Не пересоздавать интерфейс tun при переподключении. Удобно, чтобы VPN не «мертвел» при разрывах.

## `remote-cert-tls server`

Проверяет, что удалённый сервер действительно сервер, а не клиент. Усиление безопасности через сертификаты TLS.

## `cipher AES-256-CBC`

Шифр данных: AES 256 бит в режиме CBC.  
Защищает весь VPN-трафик.

verb 3

Уровень логирования OpenVPN. 0 — тихо,  
9 — суперподробно. 3 — нормальный  
уровень, показывает ошибки и  
соединения.

Вставка файлов сертификатов и  
ключей

OpenVPN позволяет вставить  
содержимое прямо в конфиг:

```
<ca>
...вставьте содержимое
ca.crt...
</ca>
```



и — блок, в который вставляется сертификат центра сертификации (CA).

- Можно вставлять полностью весь текст файла ca.crt, начиная с -----BEGIN CERTIFICATE----- и до -----END CERTIFICATE-----.

```
<cert>
...вставьте client1.crt...
</cert>
```

Сертификат клиента.

- Вставляется полностью, как есть из client1.crt.

```
<key>
...вставьте client1.key...
</key>
```

Приватный ключ клиента.

- Никому не показывать!
- Вставляется полностью, как есть из client1.key.

```
<tls-auth>
...вставьте ta.key...
</tls-auth>
```

TLS ключ для дополнительной защиты (HMAC).

- Тоже вставляется полностью из ta.key.

key-direction 1

Направление TLS-ключа (ta.key) для HMAC.

- Сервер обычно 0, клиент 1.
- Нужно, чтобы TLS-аутентификация работала корректно.

## ИТОГ по вставке файлов

Это создаёт один единый файл конфигурации, который можно просто передать клиенту. После этого OpenVPN клиенту не нужны отдельные файлы, всё уже внутри .ovpn конфига.

## Проверки после запуска

### На клиенте

На клиенте подключитесь к VPN и проверьте внешний IP (`curl ifconfig.me`) — должен быть IP сервера.

```
curl ifconfig.me
```

- 
- Разбор:
    - curl — утилита командной строки для получения данных с URL через HTTP/HTTPS.
    - ifconfig.me — сайт, который показывает ваш внешний IP.
  - Итого: команда отправляет HTTP-запрос к ifconfig.me и выводит ваш публичный IP.
  - Что проверяем:
    - Если VPN работает, IP будет IP сервера, а не IP вашей локальной сети.
    - Если IP ваш домашний/локальный — VPN не работает.

## На сервере

```
sudo iptables -t nat -L -n
-v
```

- Разбор:
  - sudo — запуск с правами администратора.
  - iptables — утилита управления пакетными фильтрами Linux.
  - -t nat — выбрать таблицу NAT (для MASQUERADE и переноса IP).
  - -L — показать все правила.
  - -n — не пытаться резолвить имена хостов и сервисов (быстрее).

- -v — подробный вывод (счётчики пакетов/байт).

- Что проверяем:

Должно быть правило MASQUERADE для сети VPN (10.8.0.0/24), чтобы трафик VPN-клиентов выходил в интернет.

```
sysctl net.ipv4.ip_forward
```

- Разбор:
  - sysctl — утилита для чтения/изменения параметров ядра Linux.
  - net.ipv4.ip\_forward — параметр ядра, отвечающий за маршрутизацию IPv4.
- Вывод:

- `net.ipv4.ip_forward = 1` — маршрутизация включена.
- `0` — выключена, VPN не сможет передавать трафик между туннелем и интернетом.

```
journalctl -u openvpn-server@server
```

- Разбор:
  - `journalctl` — утилита для просмотра логов `systemd`.
  - `-u` — выбрать логи конкретного сервиса.
  - `openvpn-server@server` — сервис OpenVPN с конфигом `server.conf`.

- Что проверяем:
  - Соединения клиентов.
  - Ошибки TLS, сертификатов, портов.

Если VPN не подключается — здесь видно причину.

```
sudo ufw status verbose
```

Рассматривали ранее

## Итог

У меня все заработало надеюсь у вас тоже. Все просто\* и понятно\*. Если что можно написать автору. А себя я поздравляю что дописал статью которую никто скорее всего никогда не прочитает.